

# TrustMeBro Client Technical Documentation

## 1) Purpose and Scope

This document is a full technical handoff for the TrustMeBro dashboard application. It is written for client teams who need to:

- run the project locally,
- understand the architecture and data flow,
- navigate the UI and API surface,
- manage environment configuration,
- troubleshoot common issues,
- and maintain or extend the codebase safely.

The current production architecture is a single **Next.js 16** app (App Router) with server route handlers and a client-side dashboard UI. For a shorter overview, see the repository `README.md`; this file is the deeper handoff.

---

## 2) System Overview

TrustMeBro is a prediction-market analytics platform that:

- aggregates live/resolved market data from PMXT (Polymarket + Kalshi),
- computes inefficiency signals (bias, divergence, liquidity mismatch, pricing mismatch),
- visualizes results in multiple charts,
- and includes an AI chat panel that reads the same dashboard context as the user.

### Runtime model

- Frontend and backend are in one codebase (Next.js).
- API logic lives in `app/api/*/route.ts`.
- UI pages live in `app/*`.
- Shared business logic/types live in `lib/*`.
- Components live in `components/*`.

### Key routes

- `/` - Landing page
  - `/dashboard` - Full analytics dashboard
-

### 3) Current Architecture

#### 3.1 High-level data flow

1. User loads `/dashboard`.
2. UI components request data from internal API routes (`/api/*`) via SWR.
3. API routes fetch/calculate data using PMXT and local analytics helpers.
4. Responses are wrapped in a common envelope and cached (HIT/MISS/BYPASS).
5. Charts render and also push selected context into the global Zustand store.
6. Chat panel sends user question + live dashboard context to `POST /api/analyst`.
7. `/api/analyst` streams response tokens back to the client.

#### 3.2 Main directories

- `app/` - Next.js App Router pages and API route handlers
  - `components/` - Dashboard UI, charts, navigation, chat, and reusable UI primitives
  - `lib/` - Type definitions, API wrappers, PMXT client, caching/rate limiting, analytics utilities
  - `tests/e2e/` - E2E tests (Playwright; `playwright.config.ts` at repo root)
  - Colocated Jest tests (e.g. `app/**/*.test.ts`, `app/**/*.test.tsx`)
  - `docs/` - Project documentation (this file)
- 

### 4) User-Facing Navigation

#### 4.1 Landing page (/)

Landing page provides:

- platform positioning,
- key feature highlights,
- and CTA button to open dashboard (`/dashboard`).

#### 4.2 Dashboard (/dashboard)

Major dashboard layout sections:

- `TopNav` - title, venue/category/date controls, theme toggle, AI panel toggle, mobile sidebar control
- `Sidebar` - jump links to chart sections (off-canvas on small viewports)
- `KPIRow` - high-level summary metrics
- Chart sections (lazy-loaded with `next/dynamic`; mounts are staggered via `DeferChartMount` to reduce burst traffic):
  - Resolution bias heatmap + resolution bias distribution (two-column row)

- Cross-venue divergence
  - Market momentum
  - Calibration curve + efficiency timeline (shared row; efficiency anchor `id="efficiency-timeline"`)
  - Liquidity gap scatter + price vs resolution (shared row; price anchor `id="price-vs-resolution"`)
  - Inefficiency leaderboard
  - `FirstTimeUserGuide` - in-page tips for reading the dashboard (`#first-time-users`)
  - `ChatPanel` - AI assistant with context visibility and streaming responses (`POST /api/analyst`)
- 

## 5) API Surface (Internal)

All API routes are implemented as Next.js route handlers and return JSON (except streaming text in chat).

Common response envelope shape:

- `data`
- `cache` (HIT | MISS | BYPASS)
- `fetchAt` (ISO timestamp)
- `source` (pmxt | archive | mock | computed)
- optional `error/meta` fields

### 5.1 GET `/api/markets`

Purpose: Retrieve markets (live or closed) with optional filters.

Query params:

- `exchange` (polymarket | kalshi)
- `category` (normalized category)
- `limit` (max 500)
- `query` (text search)
- `closed` (true/false)

Notes:

- Cache TTL: 60s live, 1h closed
- Normalizes category and venue metadata for downstream chart consistency

### 5.2 GET `/api/resolution-bias`

Purpose: Compute category x exchange NO/YES rates and z-scores from resolved markets.

Query params:

- **category** (optional single-category filter)

Notes:

- Cache TTL: 1h (resolved data)
- Returns bucket-level stats and sample metadata

### 5.3 GET /api/inefficiencies

Purpose: Compute composite inefficiency signals.

Signals:

- **resolution\_bias**
- **cross\_venue\_divergence**
- **liquidity\_gap**
- **late\_breaking\_mismatch**

Notes:

- Cache TTL: 5m
- Includes scoring and descriptive details per flagged item

### 5.4 GET /api/ohlcv

Purpose: Fetch OHLCV candles for a specific outcome.

Query params:

- **exchange** (required)
- **outcomeId** (required)
- **resolution** (default 1h)
- **limit** (1-500)

Notes:

- Cache TTL: 5m
- Falls back to mock candles when PMXT key is missing

### 5.5 GET /api/archive

Purpose: Retrieve archive rows from `archive.pmxt.dev`.

Query params:

- **path** (optional subpath)

Notes:

- Cache TTL: 24h
- Supports flexible parsing (JSON/NDJSON/plain lines)

## 5.6 POST /api/analyst

Purpose: Streaming AI assistant endpoint (dashboard chat).

Request body:

- **messages:** user/assistant message history
- **context:** dashboard context snapshot

Notes:

- Streams plain text response chunks (Vercel AI SDK + `@ai-sdk/anthropic`)
- Rate limit: **20** requests/minute/IP by default (`CHAT_RATE_LIMIT_MAX / CHAT_RATE_LIMIT_WINDOW_SECONDS` when set)
- Default model when `ANTHROPIC_MODEL` is unset: `claude-4-sonnet-20250514` (must be a dated Messages API id)
- Gracefully degrades to a mock text stream if `ANTHROPIC_API_KEY` is missing

## 5.7 Other analytical endpoints

- GET `/api/divergence` — cross-venue / category divergence payloads for charts
- GET `/api/calibration` — calibration buckets for the calibration chart
- GET `/api/efficiency-timeline` — monthly mispricing timeline
- GET `/api/warmup` — cache warmup (protect in production with `CRON_SECRET + Authorization: Bearer ...`; see `HOW_TO_DEPLOY.md`)

These support chart-specific analytics and scheduled warmup; there are **no** `/api/debug/*` routes in the current tree—use server logs and `X-Cache` headers for debugging.

---

## 6) Frontend State and Context Model

Global client state uses Zustand (`lib/store.ts`) and stores:

- active filters (`venue`, `category`, `dateRange`)
- active chart section
- currently visible markets/scores/resolution stats
- chat panel state and message stream

Why this matters:

- The chatbot does not answer in isolation.
  - It receives a live context snapshot built from what the charts have loaded.
  - This keeps AI responses aligned with visible dashboard data.
-

## 7) Caching and Rate Limiting

### 7.1 Cache backend strategy

Implemented in `lib/redis.ts`:

- Primary: Upstash Redis (`UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN`)
- Fallback: in-memory Map (for local/dev without Redis)

Behavior:

- All major PMXT-dependent endpoints use read-through caching.
- Headers include `X-Cache` for observability.
- Response envelope always includes cache state.

### 7.2 Rate limiting

- `POST /api/analyst` is protected with sliding-window rate limiting.
  - Uses Upstash `ratelimit` when configured.
  - Falls back to in-memory limiter when not configured.
- 

## 8) External Integrations

### 8.1 PMXT

Used for:

- unified market router (`/v0/markets`)
- per-venue OHLCV via `pmxtjs`

Key implementation file:

- `lib/pmxt.ts`

### 8.2 Archive endpoint

- Source: `https://archive.pmxt.dev/`
- Ingestion helper: `fetchArchive()` in `lib/pmxt.ts`

### 8.3 Anthropic

- Used by `POST /api/analyst` streaming
- Configurable model via `ANTHROPIC_MODEL` (see `.env.local.example` for valid pinned ids)

### 8.4 Upstash Redis

- Caching backend
- Chat rate-limiting backend

---

## 9) Environment Variables

Configured in `.env.local` (template in `.env.local.example`).

Required for full live behavior:

- `PMXT_API_KEY`
- `UPSTASH_REDIS_REST_URL`
- `UPSTASH_REDIS_REST_TOKEN`
- `ANTHROPIC_API_KEY`

Optional:

- `ANTHROPIC_MODEL` (defaults to `claude-4-sonnet-20250514` in code when unset)
- `PMXT_ARCHIVE_URL` (defaults to public archive URL)

Graceful degradation behavior:

- missing `PMXT` key -> deterministic mock data for market endpoints
  - missing Upstash keys -> in-memory cache/rate limit fallback
  - missing Anthropic key -> mock streaming chat response
- 

## 10) Local Development Runbook

Run from the **repository root** (not from a legacy `backend/` folder):

```
cd /path/to/trustmebro
npm install
cp .env.local.example .env.local    # optional; fill keys as needed
npm run dev
```

Open:

- `http://localhost:3001`

Useful scripts:

- `npm run dev` - start dev server
  - `npm run build` - production build
  - `npm start` - run production server
  - `npm run lint` - ESLint
  - `npm run typecheck` - TypeScript checks
  - `npm test` - unit tests
  - `npm run test:e2e` - Playwright tests
-

## 11) Migration Note: Old Two-Terminal Setup vs Current Setup

Previously, the project had separate frontend/backend directories with separate runtimes. Current architecture is unified in the root Next.js app.

Important:

- Do not rely on `backend/` as runtime entrypoint for the current version.
  - Start the app from the root `package.json`.
- 

## 12) Client Operations Guide

### 12.1 Daily health checks

- verify homepage loads (`/`)
- verify dashboard loads (`/dashboard`)
- verify API response headers include `X-Cache`
- verify the AI panel opens and `POST /api/analyst` returns a response

### 12.2 Release checks

Before deployment:

- run lint/typecheck/tests
- run build
- verify key dashboard charts populate
- verify `POST /api/analyst` for both configured and degraded modes

### 12.3 Recommended monitoring signals

- API error rate by route (`/api/*`)
  - cache hit ratio
  - PMXT upstream failures/latency
  - chat 429 rate-limit frequency
  - frontend render/runtime errors
- 

## 13) Troubleshooting Guide

### 13.1 'next' is not recognized

Cause:

- dependencies not installed in the directory being used

Fix:



1. `cd` to repository root
2. run `npm install`
3. run `npm run dev`

### 13.2 App boots but shows mock-like data

Cause:

- `PMXT_API_KEY` missing/invalid

Fix:

- set a valid PMXT key in `.env.local`

### 13.3 Chat says not configured

Cause:

- `ANTHROPIC_API_KEY` missing

Fix:

- add key and restart server

### 13.4 Cache appears to reset on restart

Cause:

- running in memory-cache fallback mode

Fix:

- configure Upstash Redis credentials

### 13.5 Too many git changes shown

Cause:

- virtual environment folders being tracked as untracked content (`.venv/`, etc.)

Fix:

- add `venv/build` artifacts to `.gitignore`
- 

## 14) Security and Reliability Considerations

- Keep API keys in `.env.local` only (never commit secrets).
- Keep rate limiting enabled for chat endpoints.
- Favor cached route usage to stay within PMXT quotas.

- Preserve graceful-degradation behavior so UX remains functional during key outages.
  - Validate and sanitize all external query parameters in any new route handlers.
- 

## 15) Extension Guide (For Future Developers)

### 15.1 Add a new chart

1. Add chart component under `components/charts/`.
2. Add API route if needed under `app/api/...`
3. Import chart dynamically in `app/dashboard/page.tsx`.
4. Wrap in `ErrorBoundary` and provide skeleton fallback.
5. Push relevant context via `updateChartContext()` if chat should reference it.
6. Add tests for data transformation/UI rendering.

### 15.2 Add a new API route

Use standard conventions:

- export `runtime = "nodejs"` and `dynamic = "force-dynamic"`
- use common response envelope (`data/cache/fetchedAt/source`)
- include `X-Cache` header when cache-backed
- include robust error handling and graceful fallback where applicable

### 15.3 Add a new inefficiency signal

1. Implement scorer in `app/api/inefficiencies/route.ts` or extracted helper.
  2. Extend `InefficiencyType/InefficiencyScore` in `lib/types.ts` if needed.
  3. Surface in `leaderboard/visualization` component.
  4. Add unit tests for `threshold/scoring` behavior.
- 

## 16) File-Level Quick Index

- `app/page.tsx` - Landing page
- `app/dashboard/page.tsx` - Dashboard shell and section composition
- `app/layout.tsx` - Global metadata/fonts/root layout
- `app/api/*/route.ts` - Internal API endpoints (streaming chat: `app/api/analyst/route.ts`)
- `components/navigation/TopNav.tsx` - Filters, theme toggle, chat toggle

- `components/navigation/ThemeToggle.tsx` - Light/dark/system theme control
  - `components/navigation/Sidebar.tsx` - Section navigation
  - `components/KPIRow.tsx` - KPI metrics and context updates
  - `components/chat/ChatPanel.tsx` - AI chat UI and stream handling (POST `/api/analyst`)
  - `components/FirstTimeUserGuide.tsx` - First-time user onboarding section
  - `lib/store.ts` - Zustand global state
  - `lib/api.ts` - Fetch envelope and refresh intervals
  - `lib/pmxt.ts` - PMXT/router/OHLCV/archive integration
  - `lib/redis.ts` - Cache + rate-limit backend abstraction
  - `lib/types.ts` - Shared app contracts and analytics types
- 

## 17) Handoff Summary

TrustMeBro is currently implemented as a single Next.js application with:

- a landing page (`/`) and dashboard (`/dashboard`),
- cache-first server routes for market analytics,
- robust fallback modes for missing third-party keys,
- and context-aware AI analysis integrated directly into the dashboard workflow (POST `/api/analyst`).

For clients, the key operational rule is simple:

- run and manage the app from the repository root (`npm run dev`), not from legacy split directories.